

مکتب شریف

اولین بوتکمپ آموزشی - استخدامی ایران



پروژه نهایی

مکتب 126





Online Store (Backend with Django, DRF, and React Frontend)

1. Overview of the Project

In this project, students will be tasked with developing the backend for an e-commerce website. They will work on functionalities such as managing products, customer accounts, orders, and discounts, while the frontend (React app) will be provided. The primary focus for students will be creating the backend APIs using Django and Django REST Framework (DRF) to handle the backend logic and connecting the React frontend to the backend.

2. Technologies Involved

- **Backend:**
 - **Django** (for the backend framework)
 - **Django REST Framework (DRF)** (to build APIs)
 - **PostgreSQL** (for database management)
 - **Redis** (for OTP generation, caching)
 - **Celery** (for background tasks like sending emails)
 - **RabbitMQ** (optional for managing task queues in Celery)
 - **JWT** (for authentication)
 - **Docker** (for containerizing the application)
 - **Nginx** (for reverse proxy and static file management)
- **Frontend:**
 - **React** (pre-built app for connecting to the backend)
 - **JavaScript, HTML, CSS** (for frontend interactions and styles)
 - **TailwindCSS or Bootstrap** (optional, for styling in React)

3. Project Breakdown (Phases) (۱۴ تیر تا ۱۹ تیر)

Phase 1: Project Setup and Initial Backend Development

- **Tasks:**
 - Analyze the project requirements and create an ERD (Entity Relationship Diagram).
 - Set up the Git repository and configure Django settings.
 - Initialize the backend with Django and DRF.
 - Set up PostgreSQL as the database.
 - Set up Redis for OTP generation and caching.
 - Develop models for:
 - Customer
 - Address
 - Product
 - Discount
 - Order
 - Order Item
 - Implement basic API views for CRUD operations on products and customers.
- **Deliverables:**
 - ERD (PDF)
 - GitHub repository with commit history.
 - Initial backend project structure.
 - Sample data in the admin panel.

Phase 2: Developing Core Features (۲۰ تیر تا ۲۶ تیر)

- **Tasks:**
 - Implement authentication with JWT (Login and Registration).
 - Implement OTP-based login via email using Redis.
 - Create product catalog and product detail APIs.

- Implement discount system with percent-based and fixed discounts.
 - Set up category models and APIs (including hierarchical categories using self relations).
 - Develop the shopping cart API for adding/removing items.
 - Implement order placement and order item management.
 - **Deliverables:**
 - Complete customer registration and login APIs.
 - Product and discount APIs.
 - Category, order, and cart APIs.
 - Working backend for connecting the frontend.
-

Phase 3: Customer and Order Management (۲۷ تیر تا ۲ مرداد)

- **Tasks:**
 - Implement functionality for managing customer profiles (view, update).
 - Implement order history and order status tracking for customers.
 - Develop API endpoints for managing orders and updating order statuses (e.g., pending, shipped, delivered).
 - Integrate background tasks using Celery for order processing (e.g., sending emails for order confirmation).
 - Set up Celery with RabbitMQ for task management.
 - **Deliverables:**
 - API endpoints for managing orders and customer data.
 - Email notifications via Celery tasks.
 - Full backend integration with the provided React frontend.
-

Phase 4: Final Refinements and Deployment (۳ مرداد تا ۹ مرداد)

- **Tasks:**
 - Final debugging and testing of the entire system.
 - Test coverage to ensure the backend APIs are well-tested (unit tests, coverage above 80%).

- Finalize JWT-based authentication and ensure token expiration/refresh logic is working.
 - Integrate the frontend (React) app with backend APIs, ensuring all interactions work seamlessly.
 - Deploy the application using Docker and Nginx (for reverse proxy and static files).
 - Prepare project documentation and README.
 - **Deliverables:**
 - Deployed project (live URL if applicable).
 - Full backend with connected frontend (via React).
 - Well-tested APIs with proper test coverage.
 - Detailed project documentation on GitHub.
-

4. Additional Project Requirements

- **Git Branching:**
 - Use develop and master branches. All code should be developed on the develop branch and merged into master after completing each phase.
- **Testing:**
 - 30% of the grade will be based on testing coverage. Ensure that unit tests are implemented for backend logic.
 - Test views and APIs using tools like **Selenium** for end-to-end testing.
 - **TDD (Test-Driven Development)** is encouraged.
- **Multi-language Support:**
 - Begin integrating translation from Phase 1 to avoid code duplication later. Ensure the backend supports different languages (for example, using Django's internationalization system).
- **Database and Models:**
 - Models should be normalized and properly related.
 - Implement **soft deletes** (delete_logical) for entities that need deletion.
- **Authentication:**
 - JWT authentication will be used for user login, and the frontend should handle storing and passing the JWT token in HTTP headers for API requests.

- **Deployment:**
 - Dockerize the application using Dockerfile and docker-compose.
 - Deploy using Nginx to serve the backend and handle static files (for production environments).
- **GitHub Setup:**
 - Ensure that the project repository on GitHub is private.
 - Add instructors as collaborators.
 - Ensure commit history is regular and properly labeled.

5. Final Project Submission Details

- Submit the final project by pushing all code to the private GitHub repository.
 - Include a detailed README.md that explains the setup, technologies, and instructions for running the backend.
 - Submit the ERD in PDF format as part of the GitHub repository.
 - Ensure that the frontend and backend are fully integrated and functional.
-

Key Adjustments for React Frontend:

1. **React Frontend:** The frontend, built in React, will be provided, and students need to ensure the backend APIs integrate seamlessly with it. Students should focus on:
 - Creating and exposing API endpoints that match the data and actions the frontend expects.
 - Handling authentication, including token management and securing routes using JWT.
 - Ensuring the backend can handle user actions like adding to the cart, checking out, viewing orders, and profile management.
2. **Frontend Backend Connection:**
 - Handle API requests in Django views with proper RESTful methods.
 - Return appropriate responses (JSON) for frontend consumption.
 - Allow CORS (Cross-Origin Resource Sharing) if the frontend is served from a different server.

